

Rammy HR Chatbot — Integration Plan & Developer Guide

Architecture Overview



The browser never talks directly to OpenAI. The API key lives only in the Python process.

Project File Structure

CSC402-Project/	
■■■ index.html	← Chat UI – includes status dot in header
■■■ chat.js	← Frontend JS – UI, API calls, history, timestamps, status dot, minimize/restore, options dropdown
■	
■■■ styling.css	← WCU-branded styles (purple/yellow)
■■■ chatbot_api.py	← Python Flask backend – all AI and retrieval logic
■■■ server.js	← Node.js Express REST API server
■■■ Dockerfile.python	← Docker image for Python service
■■■ Dockerfile.node	← Docker image for Node.js service
■■■ docker-compose.yml	← Orchestrates both containers
■■■ .env	← API keys (never commit to Git)
■■■ requirements.txt	← Python dependencies
■■■ package.json	← Node.js dependencies
■■■ INTEGRATION_PLAN.md	← This file

■■■ STARTUP.md	← Quick-start commands
■■■ README.md	← Repository overview and Docker setup
■■■ architecture.svg	← Architecture diagram (renders on GitHub)

Setup Instructions

Step 1 — Python Backend

```
cd ~/CSC402-project

# Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment
echo 'OPENAI_API_KEY=sk-your-key-here' > .env

# Start the Flask service
python chatbot_api.py
# Running on http://127.0.0.1:5001
```

Step 2 — Node.js Server

```
cd ~/CSC402-project
npm install
npm start
# Running on http://localhost:3000
```

Step 3 — Frontend

Open index.html in VS Code and use Open with Live Server, or:

```
open ~/CSC402-project/index.html
```

API Reference

POST /api/chat

Send a user message and receive Rammy's reply.

Request body

```
{
  "message": "How do I update my address?",
  "history": [
    { "role": "user",      "content": "Hello" },
    { "role": "assistant", "content": "Hi! How can I help?" }
  ]
}
```

Response

```
{ "reply": "You can update your address through Employee Self Service (ESS)..." }
```

POST /api/refresh

Triggers a background reload of all HR source pages.

```
{ "message": "Source refresh started in background." }
```

Also triggerable from the chat UI via the options dropdown (■ → Refresh HR sources) or by typing /refresh in the chat.

GET /api/health

Liveness check — also pings the Python service.

```
{ "status": "ok", "python": "reachable" }    // healthy
{ "status": "degraded", "python": "unreachable" } // Python down (HTTP 503)
```

The status dot in the chat header calls this endpoint on load and every 30 seconds.

Frontend Features

Status Dot

A small circle next to "Ask Rammy" in the header. Green = both services up, Red = one or both down. Checked every 30 seconds via /api/health.

Close Button (Minimize)

Clicking X collapses the chat to a floating purple bubble in the bottom-right corner. Clicking the bubble restores the chat. History is preserved.

Options Dropdown (■)

```
| Option | Action |
|---|---|
| Clear conversation | Wipes messages and history, shows welcome message |
| Refresh HR sources | Calls POST /api/refresh in the background |
| Contact HR | Displays clickable email and phone for WCU HR |
```

Message Timestamps

Every bubble shows the time sent (12-hour format, e.g. 2:45 PM) next to the name tag.

Clickable Source Links

Responses include HTML anchor links when the answer is partial. Links open in a new tab, styled in WCU purple. The `sanitizeHtml()` function in `chat.js` allows safe tags while blocking all other HTML.

Key Optimizations

Python (`chatbot_api.py`)

```
| Issue | Fix |
|---|---|
| client.responses.create – invalid SDK method | Changed to client.chat.completions.create |
| Sources re-fetched on every startup | Fetched once at startup, cached, refreshed on demand |
| No HTTP interface | Wrapped in Flask with /chat, /refresh, /health |
| Hardcoded "I cannot answer" reply | GPT-generated varied decline at temperature=0.8 |
| Thin keyword matching | Synonym expansion + phrase boosts for all HR topics |
| GPT not including links | Context now includes explicit "Available source URLs" list |
| Regex compiled inside functions | Pre-compiled at module level |
```

Node.js (`server.js`)

- API key never exposed to browser
- Rate limiting (60 req/min per IP)
- CORS restricted to frontend origin
- 30-second timeout on Python calls

Frontend (`chat.js`)

- Real `fetch()` replacing simulated reply
- `sanitizeHtml()` allows safe links while blocking XSS
- Input disabled while waiting (prevents double-sends)
- Conversation history maintained client-side and sent with every message

Startup Order

1. Start Docker Desktop
2. docker-compose up --build (first time) or docker-compose up
3. Open index.html in browser via Live Server

Common Issues

Symptom	Likely Cause	Fix
"chatbot backend is temporarily unavailable"	Python service not running	Check docker logs rammy-python
Status dot stays red	Either service is down	Run curl http://localhost:3000/api/health
CORS error in browser console	FRONTEND_ORIGIN mismatch	Update .env in server to match browser URL
Links not clickable	Old chat.js without sanitizeHtml	Replace with latest chat.js
Port 5000 conflict on Mac	AirPlay Receiver uses 5000	App uses 5001 – turn off AirPlay in System Settings
OPENAI_API_KEY error on startup	Missing .env	Create .env with your key in the repo root

Recommended Next Steps

1. Semantic search — Replace keyword scoring with OpenAI text-embedding-3-small embeddings for much better retrieval accuracy
2. Scheduled refresh — Add APScheduler to reload HR sources nightly automatically
3. Session IDs — Maintain conversation history server-side instead of sending it with every request
4. Cloud deployment — Deploy to Render or Railway using the existing Docker setup
5. Unicorn — Replace Flask's dev server with Unicorn for production traffic
6. React frontend — Wire up chatReact.js for response streaming (word-by-word output)
7. Rammy profile picture — Add mascot image to the chat header